GAME / SOFTWARE

COMPLETE

Rollback Netcode Layer

Deterministic lockstep networking with rollback, client-side prediction, and lag compensation for 2–8 player matches.

`Rust` `UDP` `Tokio` `Protobuf`

VERSION	STATUS	MODE	PLAYERS	TRANSPORT	UPDATED
1.2.0	Complete	Maintenance	2–8	UDP	Sep 2026

1. Overview

A rollback netcode library for deterministic lockstep games. Each client simulates forward immediately using predicted inputs for remote players; when the real inputs arrive and differ from the prediction, the simulation rolls back to the last confirmed frame and re-simulates to the present. Players see their own inputs on the next frame regardless of ping.

The library owns the transport, the input buffers, the prediction, and the rollback loop. The game supplies four things: advance one frame, save state, load state, and a state checksum. Everything else — packet redundancy, frame pacing, desync detection — sits behind that interface, which is small enough to integrate into an existing deterministic simulation in a day.

2. Frame loop

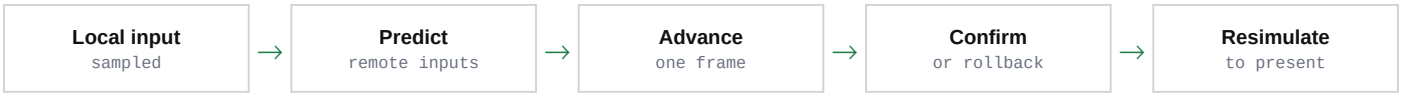

Figure 1 — Rollback cost is bounded by the prediction window; beyond it the session stalls rather than accumulating unbounded resimulation.

Every packet carries the last N frames of input rather than only the newest, so a dropped datagram is usually covered by the next one and never requires a retransmit round trip. At the default redundancy of eight frames, a session tolerates seven consecutive losses before prediction is forced to extend.

3. Components

Transport	Tokio UDP sockets with redundant input packing, sequence numbers, and NAT keepalive.
Input buffer	Per-player ring of confirmed and predicted inputs with a configurable delay.
Predictor	Repeats the last known input by default; a game-supplied predictor can override.
State ring	Fixed pool of saved frames using the game's save and load hooks, no allocation in the loop.
Rollback controller	Detects misprediction, restores the confirmed frame, and re-simulates headlessly.
Desync detector	Per-frame checksums exchanged and compared; mismatch halts with a frame report.

4. Integration

```
impl GameSession for Brawler {
    fn advance(&mut self, inputs: &[Input; MAX_PLAYERS]) { self.step(inputs); }
    fn save(&self)  -> Vec<u8> { self.serialize_state() }
    fn load(&mut self, s: &[u8]) { self.restore_state(s); }
    fn checksum(&self) -> u64 { self.state_hash() }
}

let mut net = Session::builder()
    .players(4)
    .input_delay(2)              // frames; trades latency for fewer rollbacks
    .prediction_window(8)        // max frames simulated ahead of confirmation
    .redundancy(8)               // past inputs repeated per packet
    .checksum_interval(30)       // frames between desync comparisons
    .build(transport)?;

loop {
    net.add_local_input(pad.sample());
    net.tick(&mut game)?;        // advances, rolls back, and re-simulates as needed
}
```

Listing 1 — The library never renders and never sleeps; frame pacing stays with the game loop.

5. Determinism requirements

- **Fixed point or controlled floats.** The simulation must produce identical results on every target. Cross-platform float determinism is achievable but fragile; fixed point is recommended.
- **Complete state.** Save and load must cover everything that affects simulation, including RNG state and timers. A field omitted from save is the single most common source of desync.
- **No wall clock.** Simulation must depend only on the frame counter and inputs. Any read of real time, system entropy, or GPU state breaks rollback.

6. Measured behaviour

DIMENSION	MEASURED	CONDITIONS
Rollback cost	0.4 ms per frame rewound	2 KB state, 4 players, typical 2–3 frames.
Bandwidth	7.2 KB/s per peer	60 Hz, 8-frame redundancy, 8-byte inputs.
Loss tolerance	Seamless to 7% loss	Degrades to visible hitching beyond 12%.
Session ceiling	8 players, 180 ms RTT	Beyond this the prediction window dominates.

7. Operational notes

- A desync halts the session and writes a report containing the diverging frame, both checksums, and the input history, which is normally enough to locate the offending system without a repro.
- Input delay is exposed as a runtime setting rather than a compile-time constant; two frames suits most matches, and a rollback-heavy connection is better served by three than by none.
- Peer-to-peer topology is assumed. There is no authoritative server mode and no anti-cheat; both are out of scope for a deterministic lockstep model.

8. Maintenance status

AREA	STATE	COMMITMENT
Feature work	Closed	The 1.x protocol is final; no wire format changes planned.
Bug fixes	Active	Correctness issues fixed and released as patch versions.
Dependencies	Active	Tracks stable Tokio and Rust releases.